

# Sweet Medical - Plataforma de Seguro de la Salud



Trabajo Práctico Integrador  
-1C 2026-

## Contexto general

Durante los últimos años, la digitalización de los servicios de salud ha cobrado un rol central en la mejora del acceso, la organización y la calidad de la atención médica. La posibilidad de gestionar turnos en línea, consultar cartillas médicas, recibir recordatorios automáticos y coordinar agendas de atención de manera eficiente se ha vuelto una necesidad tanto para pacientes como para profesionales de la salud.

En este contexto, médicos y centros de atención requieren plataformas que les permitan administrar de forma ordenada sus horarios, disponibilidad, sedes de atención y los servicios que ofrecen, sin recurrir a procesos manuales o herramientas desconectadas entre sí. A su vez, los pacientes demandan soluciones simples e intuitivas para buscar profesionales según su obra social, especialidad o ubicación, y **reservar turnos** de manera rápida y confiable.

Como respuesta a esta necesidad, una organización del sector salud ha solicitado el desarrollo de **Sweet Medical**, una plataforma web orientada a la gestión de turnos médicos. El sistema permitirá a los profesionales definir su disponibilidad y los servicios que brindan, mientras que los pacientes podrán consultar la cartilla de acuerdo a su cobertura y solicitar turnos disponibles.

Dependiendo del plan de la obra social del paciente, algunas prestaciones estarán totalmente cubiertas, otras parcialmente cubiertas y otras no cubiertas, lo que impactará directamente en el costo final del turno reservado. El sistema deberá contemplar estas reglas de cobertura al momento de la reserva.

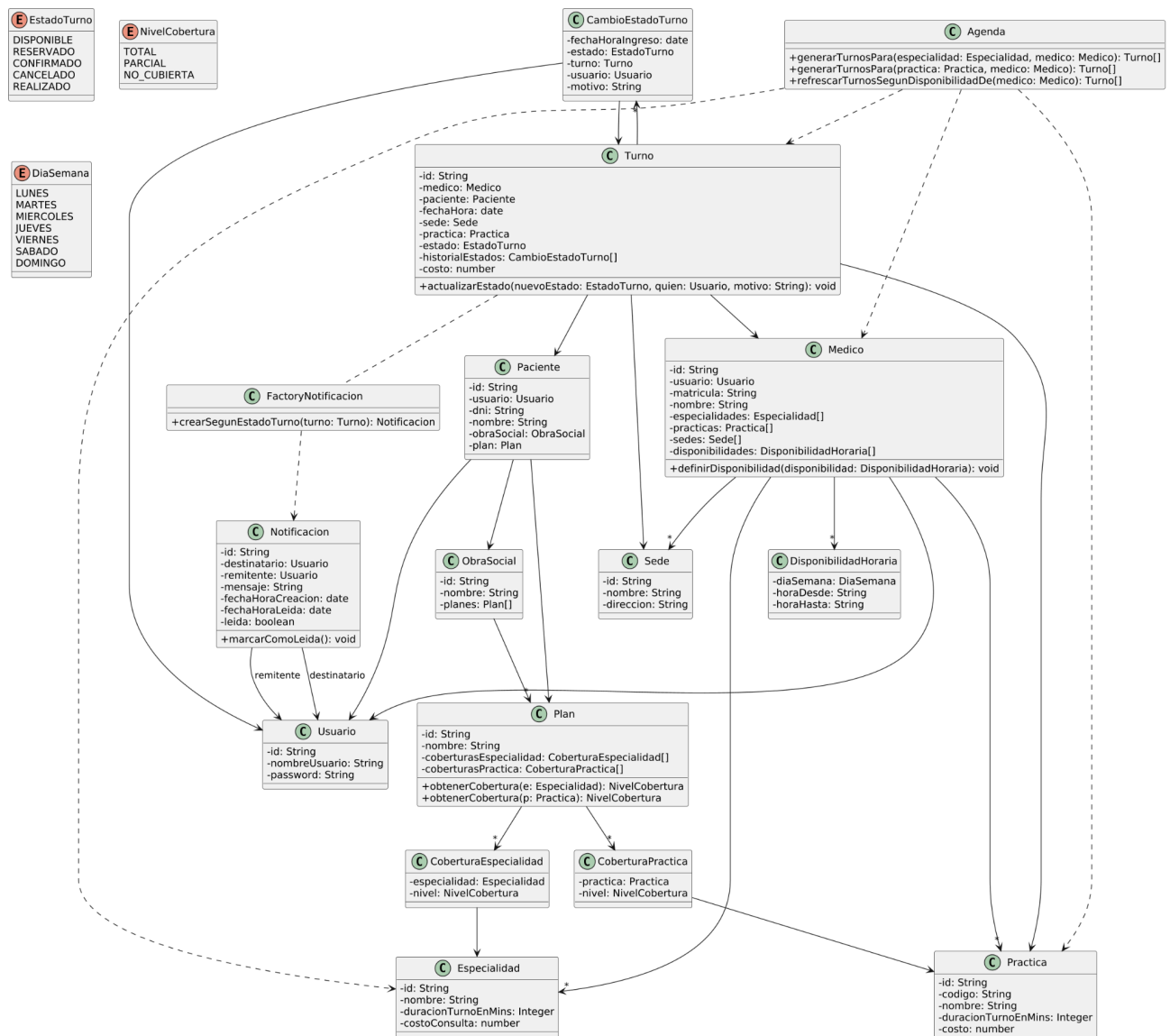
Además, la plataforma contará con un sistema de notificaciones que mantendrá informados a pacientes y profesionales ante eventos relevantes del ciclo de vida de un turno, tales como solicitudes, confirmaciones, cancelaciones, modificaciones de fecha y recordatorios previos a la atención.

Para llevar adelante este objetivo, diseñaremos e implementaremos **Sweet Medical**, una solución web moderna y escalable. El desarrollo del sistema se organizará en distintas iteraciones, cada una enfocada en un conjunto específico de funcionalidades clave, permitiendo evolucionar progresivamente la solución desde el modelo de dominio hasta su puesta en producción.

## Entrega 1: Implementación del Modelo de Objetos + Configuración del Proyecto

### Contexto

En esta primera iteración nos enfocaremos en implementar el diseño del modelo de objetos provisto por el Teach Leader, que por cierto consideramos correcto. Será nuestra responsabilidad implementar cada uno de los métodos mencionados en el Diagrama de clases.



Respecto a las notificaciones, nuestro Teach Leader nos ha mencionado que:

- Al reservar un turno, se notifica al médico indicando paciente y servicio solicitado (especialidad o práctica).
- Al aceptar un turno, se notifica al paciente.

- Ante cancelaciones de turnos, se notifica a la contraparte correspondiente.
- El día previo al turno, se envía un recordatorio tanto al paciente como al médico.

Por otra parte, también debemos implementar un primer endpoint que nos permita verificar el estado de salud de nuestro Sistema (el famoso “/health”).

### Entregables

1. **Implementación** del Diagrama de Clases provisto, completando cada uno de los métodos mencionados y agregando aquellos que el equipo considere necesarios.
2. **Implementación** del endpoint “Health Check”.
3. **Explicación** del Git flow definido para utilizar a lo largo del proyecto.

### Tecnologías a utilizar

- Backend
  - Lenguaje de programación base: [JavaScript](#)
  - Entorno de ejecución: [Node.js](#)
  - Framework: [Express](#)
- Git como Sistema de Versionado de Código, con repositorios en Github.

## Entrega 2: Exposición de APIs + Persistencia

### Contexto

En esta segunda iteración debemos desarrollar los siguientes endpoints necesarios para garantizar el cumplimiento de los requerimientos listados a continuación, teniendo en cuenta que todos los datos que se manipulen deben ser persistentes. Además, nuestro Teach Leader nos ha solicitado que la API se base en el enfoque REST.

### Gestión de Turnos

La plataforma debe permitir la **reserva de turnos** por parte de los pacientes. Esto implica la implementación de endpoints que gestionan el ciclo de vida de un turno.

### **Disponibilidad de Médicos**

- Cada médico define su disponibilidad horaria.
- A partir de dicha disponibilidad, el Sistema genera turnos en estado DISPONIBLE.
- La generación de turnos debe realizarse de forma asincrónica, mediante un proceso batch.
- Si un médico modifica su disponibilidad:
  - Los turnos existentes con fecha previa a la actual no se modifican.
  - Los turnos existentes RESERVADOS con fecha posterior a la actual, no se modifican.
  - El cambio impacta únicamente en la generación de turnos futuros y para turnos existentes futuros pero en estado DISPONIBLE.

### **Pacientes**

- Reservar un turno validando disponibilidad.
- Cancelar un turno con al menos 1 hora de anticipación, indicando un motivo.
- Consultar historial personal de turnos.
- Solicitar cambio de fecha de un turno (requiere confirmación).

### **Médicos**

- Cancelar turnos con al menos 1 hora de anticipación, indicando un motivo.
- Consultar historial de turnos de un paciente.
- Proponer modificación de fecha de un turno (requiere confirmación).
- Marcar un turno como realizado.
- Consultar su disponibilidad para una especialidad o práctica.

### Gestión de Servicios

Los médicos administran los servicios que ofrecen, entiendo por “servicios” sus especialidades y/o prácticas que realizan:

- Alta de servicios.
- Modificación de servicios.
- Baja de servicios.

### Búsqueda de turnos

La plataforma deberá permitir a los pacientes realizar la **búsqueda de turnos disponibles**, teniendo en cuenta tanto criterios médicos como las condiciones de cobertura del paciente.

La búsqueda de turnos deberá soportar, como mínimo, los siguientes filtros:

- Profesional (médico específico).
- Especialidad.
- Práctica.
- Sede de atención.
- Rango de fechas.

Al momento de realizar la búsqueda, el Sistema deberá considerar la **obra social y el plan** del paciente, de modo de poder calcular y mostrar, para cada turno disponible:

- Si la prestación se encuentra cubierta, parcialmente cubierta o no cubierta.
- El **monto que el paciente deberá abonar** en caso de reservar dicho turno.

Los resultados de búsqueda deberán mostrar información relevante del turno, incluyendo:

- Profesional.
- Especialidad y/o práctica asociada.
- Fecha, hora y sede.
- Estado del turno (siempre DISPONIBLE en este contexto).
- Costo estimado para el paciente según su plan.

Por otra parte, también se espera:

- Implementación de paginación para mejorar la eficiencia de las consultas.
- Ordenamiento por costo y fecha ascendente/descendente

### Visualización de notificaciones

Los usuarios deben recibir información relevante a través de notificaciones dentro de la plataforma.

Se espera:

- Endpoint para obtener la lista de notificaciones sin leer de un usuario.
- Endpoint para obtener la lista de notificaciones leídas de un usuario.

- Endpoint para marcar una notificación como leída.

Por otro lado, también es necesario implementar los tests unitarios sobre la capa de Servicios del Sistema.

### Entregables

1. **Implementación** de la API REST del Sistema, que incluya todos los endpoints necesarios para dar solución a los requerimientos listados.
2. **Implementación** de la persistencia de las entidades de dominio en una base de datos NoSQL Documental.
3. **Implementación** de los Test Unitarios de la capa de servicios y de la capa de dominio.
4. **Documentación** de la API REST.

### Tecnologías a utilizar

- Todas las mencionadas en la primera iteración.
- [MongoDB](#) como Base de Datos Documental NoSQL.
- [Jest](#) como framework de Testing Unitario.
- [Swagger](#) como Herramienta de Documentación de APIs. Está permitido la utilización de alguna dependencia que genere el contenido de Swagger.

## Entrega 3: UI + Integración con Backend

### Contexto

En esta tercera iteración nos enfocaremos en el desarrollo inicial del Frontend de nuestro aplicativo. En particular, trabajaremos en diseñar, maquetar e implementar las pantallas, con la correspondiente navegabilidad entre ellas, pero solamente integrando una de las funcionalidades (por ahora) con nuestro Backend.

Nuestro Teach Leader nos ha dicho que tenemos la libertad de generar las interfaces de usuario según nuestro gusto y criterio, pero teniendo en cuenta que es importante respetar los siguientes requerimientos no funcionales:

#### Interfaz Intuitiva

- El diseño de la aplicación debe ser claro y fácil de entender para usuarios sin experiencia previa en plataformas similares.
- Se debe utilizar una estructura de navegación coherente, asegurando que los usuarios puedan acceder a las funcionalidades principales en pocos clics.

#### Aprendizaje Rápido

- La interfaz debe seguir patrones de diseño comunes en aplicaciones de ecommerce para que los usuarios reconozcan fácilmente las funcionalidades.
- Se deben incluir textos descriptivos y elementos visuales que guíen a los usuarios en cada paso del proceso de reserva de turnos.

#### Feedback Visual y Notificaciones

- Deben incluirse mensajes de error y confirmación en cada interacción relevante.
- Indicadores visuales (loaders, skeletons, etc.) deben mostrar el estado de carga de las solicitudes al Backend para evitar que el usuario piense que la aplicación está inactiva.

#### Diseño Responsivo

- El aplicativo debe ser completamente funcional en distintos dispositivos (desktop, tablets y móviles).
- Se debe asegurar que todos los elementos sean accesibles en pantallas de distintos tamaños sin afectar la usabilidad.



### Accesibilidad

- Se deben seguir buenas prácticas de accesibilidad, como el uso de contrastes adecuados, etiquetas ARIA y soporte para navegación mediante teclado.
- Los textos y botones deben ser lo suficientemente grandes para facilitar su uso en dispositivos móviles.

### Consistencia en la UI

- Los estilos, colores y tipografías deben ser homogéneos en toda la aplicación para mejorar la experiencia del usuario.
- Los componentes reutilizables deben seguir un diseño coherente para evitar confusión.

Por otro lado, tal como fue mencionado, tendremos que implementar de forma completa la funcionalidad de “*Búsqueda y Visualización de Servicios*”, realizando las correspondientes llamadas a nuestro Backend.

Además, se deberá implementar un módulo similar conceptualmente a un carrito, pero orientado a la **preselección de turnos**, que permita a los usuarios:

- Permite al paciente preseleccionar uno o varios turnos disponibles.
- Visualizar un resumen de los turnos preseleccionados.
- Eliminar turnos de la preselección.

***Este módulo no tiene persistencia en backend y vive exclusivamente en el frontend.***

### Entregables

1. **Maquetado** de todas las pantallas que el equipo considere necesarias para dar cumplimiento a todos los requerimientos funcionales de la iteración 2, con la correspondiente navegabilidad entre ellas y ajuste al cumplimiento de los requerimientos no funcionales listados en esta iteración.
2. **Implementación** de funcionalidad completa de “*Búsqueda de turnos*”, incluyendo su integración con el backend.
3. **Justificación** acerca del cumplimiento de los requerimientos no funcionales.

### Tecnologías a utilizar

- Todas las mencionadas en las anteriores entregas.
- [Next.js](#) como Framework para el desarrollo de nuestro Frontend.
- [React](#) como biblioteca de generación de componentes para nuestro Frontend.
- [Axios](#) como Cliente HTTP para integrar nuestro Frontend con nuestro Backend.

## Entrega 4: Entrega final

### Contexto

En esta última iteración nos enfocaremos en terminar de implementar todas las funcionalidades de nuestro aplicativo, integrando de forma completa el Frontend con el Backend. Además, también tendremos que ocuparnos de realizar Tests de Integración en la capa de controladores de nuestro Backend, y Test E2E en nuestro Frontend.

Por último, nuestro Teach Leader nos ha indicado que llegó el momento de poner en producción la primera versión del aplicativo para que esté disponible en la Web para todos los usuarios, motivo por el cual tendremos que ocuparnos de desplegar la solución en nube.

### Entregables

1. **Implementación** de todas las funcionalidades completas, con la correspondiente integración con el Backend.
2. **Despliegue** del aplicativo en la nube (Backend + Frontend).
3. **Documentación** acerca del despliegue del aplicativo que contenga el detalle de los pasos a seguir para desplegar el aplicativo por primera vez y por cada vez que se quiere subir a producción una nueva release.
4. **Tests de integración** en la Capa de Controladores del Backend: solamente es necesario realizar 1 test, cuyo endpoint a testear queda a criterio del equipo.
5. **Tests E2E** en el Frontend: solamente es necesario realizar 1 test, cuya funcionalidad a testear queda a criterio del equipo.

### Tecnologías a utilizar

- Todas las mencionadas en las anteriores entregas.
- [Render](#) como Cloud Application Platform para el despliegue de nuestro Backend (se pueden utilizar algunas otras alternativas, a criterio del equipo y en común acuerdo con el equipo docente).
- [Netlify](#) como plataforma para despliegue de nuestro Frontend (se pueden utilizar algunas otras alternativas, a criterio del equipo y en común acuerdo con el equipo docente).
- [Cypress](#) como herramienta de Testing E2E. Está permitido buscar y utilizar alguna otra herramienta alternativa (como Jest, por ejemplo).
- [Jest](#) como framework para Testing de Integración